Adviana Kirubalin

MiniProject5 - Physics-Inspired Machine Learning and Optimization

Introduction

For this project, I explored the principle of simulated annealing from physics-inspired optimization and how it can be applied to solving complex optimization problems. The main idea behind simulated annealing is based on the physical process of cooling materials, where a system starts at a high temperature and gradually cools down to reach a stable low energy state. In the algorithm, this is modeled by starting with random solutions and allowing both better and sometimes worse solutions to be accepted, especially at higher temperatures, which helps the algorithm explore different regions of the search space.Instead of always choosing only better solutions like greedy methods, simulated annealing allows controlled randomness so that it does not get stuck too early in local minima, which is important for problems with many possible suboptimal solutions.

The key property being explored in this project is how the cooling schedule affects the ability of the algorithm to find near-optimal solutions, and how the balance between exploration and convergence changes over time. Because the algorithm includes randomness in both the starting point and the acceptance of worse solutions, it is stochastic. For this reason, it is important to run the algorithm multiple times and observe both the average performance and variation between runs. This project focuses on understanding how simulated annealing behaves under different cooling rates and how parameter choices influence the overall optimization results.

Methods

After introducing the main idea of simulated annealing, I designed a controlled experiment to explore how the cooling schedule and randomness influence optimization performance. I focused on implementing the algorithm step by step and changing its parameters to observe how its behavior changes. The goal was to understand how the algorithm balances exploration and convergence, and how sensitive it is to the cooling rate parameter.

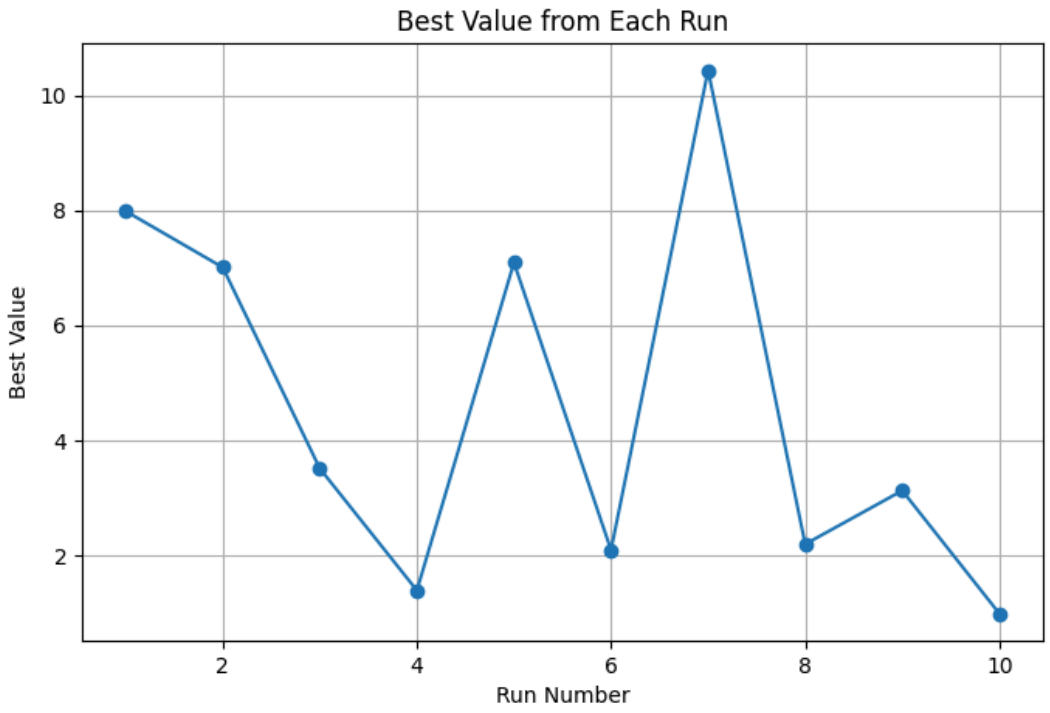
I implemented simulated annealing from scratch in Python without using external optimization libraries. The objective function used in this experiment was the Rastrigin function in two dimensions, which is known to have many local minima. This makes it a good test case because simple methods can easily get stuck in suboptimal solutions. The algorithm begins by selecting a random starting point (x, y) within a bounded search space. At each iteration, a new candidate solution is generated by making a small random move from the current point. The objective value of this new solution is then compared to the current one.

If the new solution has a lower objective value, it is accepted immediately. If the new solution is worse, it may still be accepted with a probability that depends on the difference in objective value and the current temperature. This probability is computed using an exponential function, which allows worse solutions to be accepted more frequently at higher temperatures. The temperature is gradually reduced over time using a cooling schedule, which controls how quickly the algorithm transitions from exploration to more stable convergence behavior.

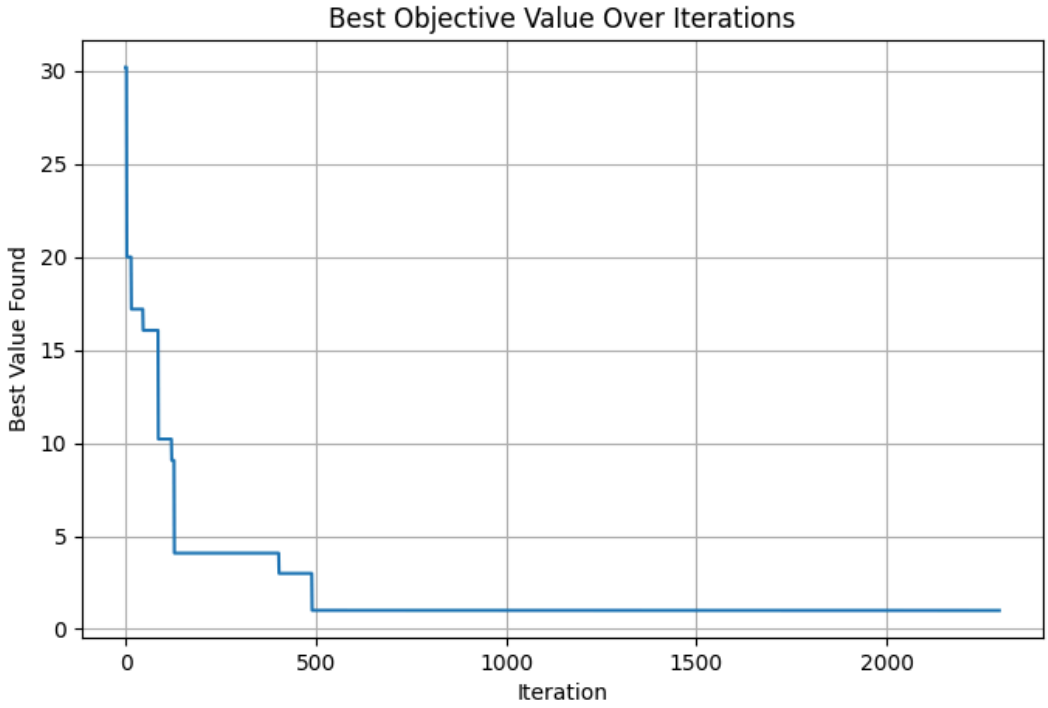
To evaluate the performance of the algorithm, I ran multiple trials because the results depend on random initialization and stochastic decisions during the search process. For each run, I recorded the best objective value found. I repeated the experiment multiple times for each cooling rate and computed summary statistics such as average, best, and worst values across runs. This allowed me to observe variability in performance and identify consistent patterns.

I also tested multiple cooling rates (0.99, 0.995, and 0.999) to examine how the rate of temperature decrease affects the algorithm's ability to find near optimal solutions. Faster cooling rates reduce temperature quickly, which can cause the algorithm to converge early and get stuck in local minima. Slower cooling rates allow more exploration, increasing the chances of escaping local minima but requiring more iterations. By comparing results across these different settings and visualizing them using graphs, I was able to analyze how parameter choices influence the effectiveness of simulated annealing.

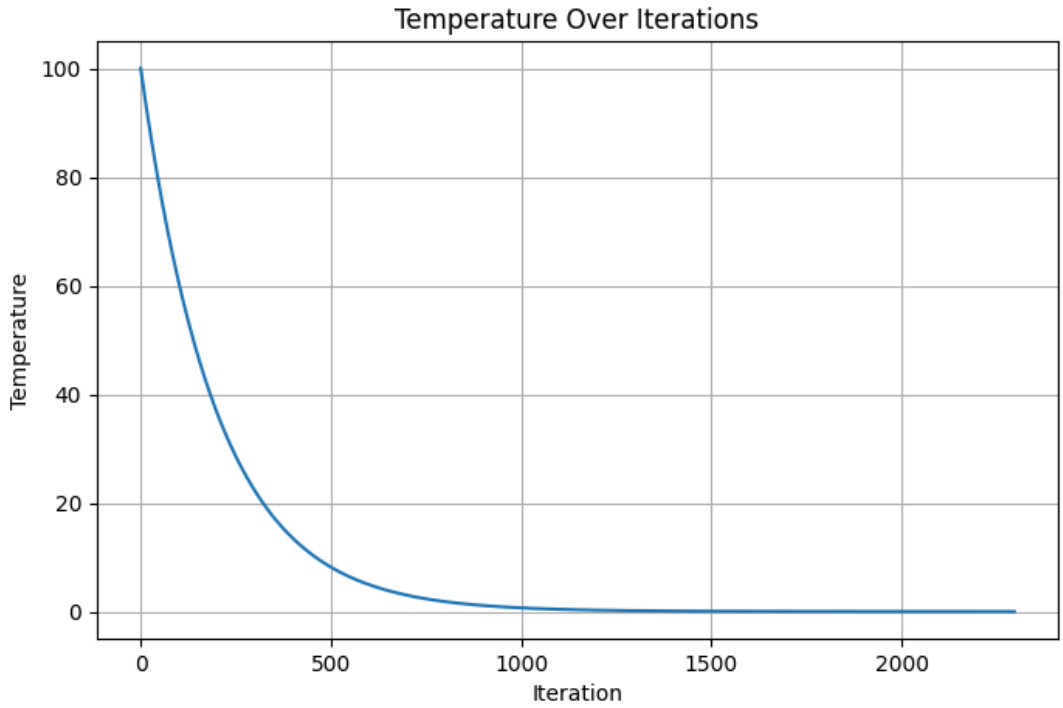
Results



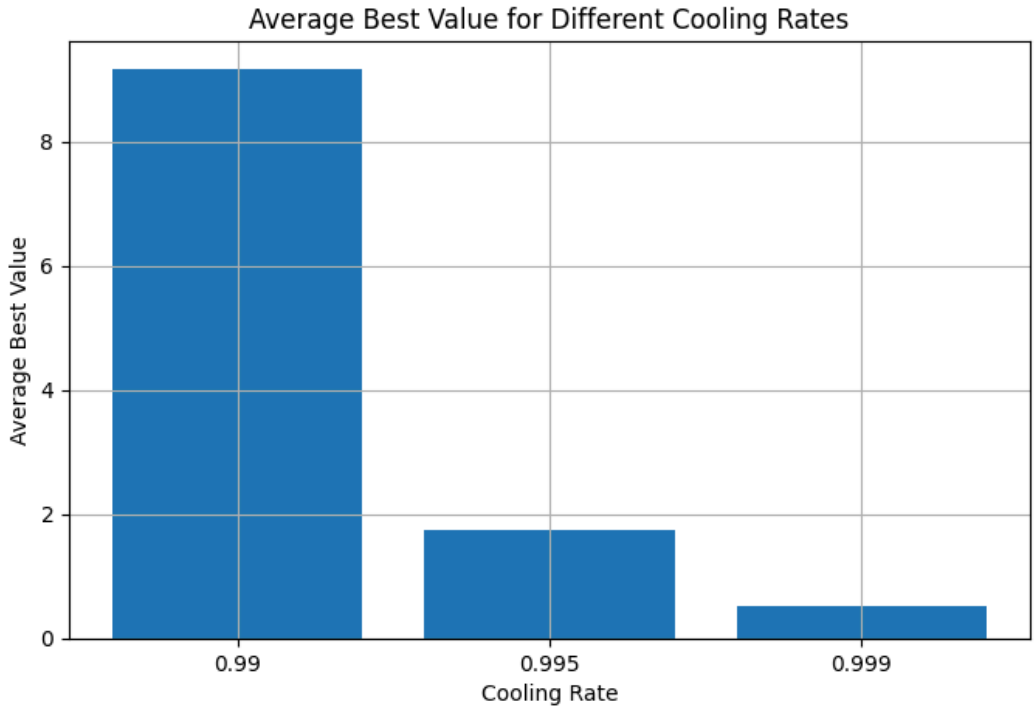
The plotted best values from each run appear spread across different levels while still generally staying in the lower range of the objective function. This shows that the simulated annealing process is able to consistently reduce the objective value from higher starting points, even though the final results vary between runs. Some runs reach very low values close to the global optimum, while others remain higher, which indicates that the algorithm sometimes gets stuck in local minima depending on the search path. The variation in values across runs lies in regions where randomness plays a role, which indicates that the algorithm's stochastic behavior directly affects the quality of the final solution. The overall distribution of best values demonstrates that the algorithm is functioning as intended by exploring the search space and finding improved solutions while balancing randomness and convergence.



The best objective value appears to decrease sharply at the beginning of the iterations while then gradually leveling off as the algorithm continues. This shows that the simulated annealing process is able to quickly improve the solution when exploration is high at higher temperatures. The later part of the graph lies in regions where the value changes very little, which indicates that the algorithm has reached a stable solution and is no longer finding better candidates. The flattening of the curve demonstrates that the system has cooled down and convergence has occurred. The overall trend demonstrates that the algorithm is functioning as intended by improving rapidly at first and then stabilizing near a low objective value.



The temperature appears to decrease rapidly in the early iterations while then gradually approaching zero as the algorithm continues. This shows that the simulated annealing process begins with a high temperature that allows more exploration of the search space. The later part of the curve lies in regions where the temperature changes very little, which indicates that the system has cooled down and is no longer allowing significant exploration. The smooth decline of the curve demonstrates that the cooling schedule is being applied consistently throughout the iterations. The overall trend demonstrates that the algorithm is functioning as intended by gradually reducing randomness and shifting from exploration to convergence.



Discussion

The average best values appear to differ significantly across the different cooling rates while showing a clear trend as the rate changes. This shows that the simulated annealing process is strongly affected by how quickly the temperature decreases during the search. The lower bar for the slower cooling rate lies in a region with much smaller objective values, which indicates that the algorithm is able to find better solutions when given more time to explore. The higher bar for the faster cooling rate demonstrates that the algorithm converges too quickly and gets stuck in worse solutions. The overall comparison demonstrates that the algorithm is functioning as intended by showing that slower cooling improves optimization performance.

The results show that the simulated annealing algorithm successfully moves toward better solutions over time while handling the complexity of the search space. The best value plots confirm that the algorithm quickly improves in the early iterations and then gradually stabilizes, which demonstrates how the cooling process helps refine the solution. The temperature graph shows a smooth decrease, indicating that the algorithm transitions from exploration to exploitation as intended. The variation across multiple runs is noticeable but not extreme, which suggests that although the process is random, it still consistently finds reasonably good solutions.

The experiments with different cooling rates highlight the importance of the temperature schedule parameter. Faster cooling rates reduce exploration and often lead to worse final

solutions, while slower cooling rates allow the algorithm to search more thoroughly and achieve lower objective values. This trade off explains a key property of simulated annealing performance depends on how carefully the cooling schedule is chosen. Overall, the results demonstrate that simulated annealing is effective at escaping local minima and improving solutions over time, but that proper parameter tuning is necessary for reliable optimization performance.

Appendix - main.py

```
import math
import random
import matplotlib.pyplot as plt

def rastrigin(x, y):
return 20 + (x * x - 10 * math.cos(2 * math.pi * x)) + (y * y - 10 * math.cos(2 * math.pi * y))

def get_neighbor(x, y, step_size):
new_x = x + random.uniform(-step_size, step_size)
new_y = y + random.uniform(-step_size, step_size)
new_x = max(-5.12, min(5.12, new_x))
new_y = max(-5.12, min(5.12, new_y))

return new_x, new_y

def simulated_annealing(cooling_rate=0.995):
current_x = random.uniform(-5.12, 5.12)
current_y = random.uniform(-5.12, 5.12)
current_value = rastrigin(current_x, current_y)

best_x = current_x
best_y = current_y
best_value = current_value

temperature = 100.0
min_temperature = 0.001
step_size = 0.5
max_iterations = 5000
best_values = []
temperatures = []
```

```
for _ in range(max_iterations):
if temperature < min_temperature:
break

new_x, new_y = get_neighbor(current_x, current_y, step_size)
new_value = rastrigin(new_x, new_y)

delta = new_value - current_value

if delta < 0:
current_x = new_x
current_y = new_y
current_value = new_value
else:
probability = math.exp(-delta / temperature)
if random.random() < probability:
current_x = new_x
current_y = new_y
current_value = new_value

if current_value < best_value:
best_x = current_x
best_y = current_y
best_value = current_value

best_values.append(best_value)
temperatures.append(temperature)
temperature = temperature * cooling_rate

return best_x, best_y, best_value, best_values, temperatures

def run_multiple_times(num_runs=10, cooling_rate=0.995):
all_best_values = []
last_best_values = None
last_temperatures = None

print(f"\nRunning simulated annealing {num_runs} times - ")
print(f"Cooling rate = {cooling_rate}")
print("-" * 40)
```

```
for i in range(num_runs):
best_x, best_y, best_value, best_values, temperatures = simulated_annealing(cooling_rate)

print(f"Run {i + 1}")
print(f"x = {best_x:.4f}")
print(f"y = {best_y:.4f}")
print(f"f(x, y) = {best_value:.6f}")
print("-" * 40)

all_best_values.append(best_value)
last_best_values = best_values
last_temperatures = temperatures

average_value = sum(all_best_values) / len(all_best_values)
best_value = min(all_best_values)
worst_value = max(all_best_values)

print("\nFINAL SUMMARY")
print(f"Average best value: {average_value:.6f}")
print(f"Best value: {best_value:.6f}")
print(f"Worst value: {worst_value:.6f}")

return all_best_values, last_best_values, last_temperatures

def compare_cooling_rates():
cooling_rates = [0.99, 0.995, 0.999]
averages = []

print("\nCOMPARING COOLING RATES")
print("=" * 40)

for rate in cooling_rates:
results, _, _ = run_multiple_times(num_runs=5, cooling_rate=rate)
avg = sum(results) / len(results)
averages.append(avg)
print(f"Cooling rate {rate} average best value = {avg:.6f}")
print("=" * 40)

plt.figure(figsize=(8, 5))
plt.bar([str(rate) for rate in cooling_rates], averages)
```

```
plt.title("Average Best Value for Different Cooling Rates")
plt.xlabel("Cooling Rate")
plt.ylabel("Average Best Value")
plt.grid(True)
plt.show()

def main():
all_best_values, best_values, temperatures = run_multiple_times(num_runs=10,
cooling_rate=0.995)
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(all_best_values) + 1), all_best_values, marker="o")
plt.title("Best Value from Each Run")
plt.xlabel("Run Number")
plt.ylabel("Best Value")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(best_values)
plt.title("Best Objective Value Over Iterations")
plt.xlabel("Iteration")
plt.ylabel("Best Value Found")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(temperatures)
plt.title("Temperature Over Iterations")
plt.xlabel("Iteration")
plt.ylabel("Temperature")
plt.grid(True)
plt.show()
compare_cooling_rates()

if __name__ == "__main__":
main()
```